

Estrutura de Dados\avaliacao_haroldo_e jose_carlos\pilha.py

```
1  from nodo import Nodo
2
3
4  class Pilha:
5      """
6      Implementação de Pilha usando lista encadeada.
7      Regra: LIFO - Last In, First Out.
8      """
9
10     def __init__(self):
11         self.topo = None
12
13     def vazia(self):
14         return self.topo is None
15
16     def push(self, info):
17         novo = Nodo(info)
18         novo.prox = self.topo
19         self.topo = novo
20
21     def pop(self):
22         if self.vazia():
23             return None
24
25         aux = self.topo
26         self.topo = self.topo.prox
27         info = aux.info
28         del aux
29
30         return info
31
32     def top(self):
33         if self.vazia():
34             return None
35
36         return self.topo.info
37
38     def imprimir(self):
39         if self.vazia():
40             print("Pilha vazia")
41             return
42
43         aux = Pilha()
44
45         while not self.vazia():
46             elemento = self.pop()
47             print(elemento)
48             aux.push(elemento)
49
50         while not aux.vazia():
51             self.push(aux.pop())
52
53
```

```

54 def buscar(self, info):
55     """
56     Busca um elemento na pilha.
57     Retorna True se encontrar e False caso contrário.
58     """
59     aux = self.topo
60
61     while aux:
62         if aux.info == info:
63             return True
64
65         aux = aux.prox
66
67     return False
68
69
70 def editar(self, info, novo_info):
71     """
72     Edita o primeiro elemento encontrado na pilha.
73     Substitui info por novo_info.
74     """
75     aux = Pilha()
76     editado = False
77
78     while not self.vazia():
79         elemento = self.pop()
80
81         if elemento == info and not editado:
82             aux.push(novo_info)
83             editado = True
84         else:
85             aux.push(elemento)
86
87     while not aux.vazia():
88         self.push(aux.pop())
89
90     if editado:
91         print("Elemento editado com sucesso")
92     else:
93         print("Elemento não encontrado")
94
95 def remover(self, info):
96     """
97     Busca e remove o primeiro elemento encontrado na pilha.
98     """
99     aux = Pilha()
100     removido = False
101
102     while not self.vazia():
103         elemento = self.pop()
104
105         if elemento == info and not removido:
106             removido = True
107         else:
108             aux.push(elemento)

```

```
109
110     while not aux.vazia():
111         self.push(aux.pop())
112
113     if removido:
114         print("Elemento removido com sucesso")
115     else:
116         print("Elemento não encontrado")
117
118 def remover_repetidos(self):
119     """
120     Remove valores repetidos da pilha.
121     Mantém apenas a primeira ocorrência encontrada a partir do topo.
122     """
123     aux = Pilha()
124     valores = []
125
126     while not self.vazia():
127         elemento = self.pop()
128
129         if elemento not in valores:
130             valores.append(elemento)
131             aux.push(elemento)
132
133     while not aux.vazia():
134         self.push(aux.pop())
135
136     print("Valores repetidos removidos")
137
```